Lab # 05

Vending Machine Design using FSM in VHDL

Digital System Design

Name: Muhammad Ahmad
Roll No: FA23-BCE-113
Section: BCE-6A
Date: April 2, 2026

Department of Computer Engineering
HITEC University, Taxila

Contents

1 Introduction to FSM-Based Control **3**
1.1 Why FSM for Complex Systems? . . . 3
1.2 Moore Machine Characteristics . . . 3

2 In-Lab Task 1: Multi-Mode Complex Counter (Moore FSM) **4**
2.1 Problem Statement . . . 4
2.2 Understanding the Problem . . . 4
2.2.1 What is Gray Code? . . . 4
2.3 Design Approach - Why Moore Machine? . . . 4
2.4 FSM State Diagram - Binary Mode (m=0) . . . 5
2.5 FSM State Diagram - Gray Code Mode (m=1) . . . 5
2.6 Combined FSM State Diagram . . . 6
2.7 Step-by-Step: How the FSM Works . . . 6
2.7.1 Example 1: Binary Mode (m=0) . . . 6
2.7.2 Example 2: Gray Mode (m=1) . . . 6
2.7.3 Example 3: Switching Modes Mid-Count . . . 7
2.8 State Transition Table . . . 7
2.9 Gray Code Sequence - Detailed Explanation . . . 7
2.10 VHDL Implementation . . . 8
2.11 Testbench . . . 9

3 In-Lab Task 2: Vending Machine Design (Moore FSM) **11**
3.1 Understanding the Problem . . . 11
3.2 Machine Specifications . . . 11
3.2.1 Accepted Coins . . . 11
3.2.2 Available Drinks . . . 11
3.2.3 Machine Rules . . . 11
3.3 System Block Diagram . . . 12
3.4 Why Moore Machine? . . . 12
3.5 State Design - The Heart of the FSM . . . 12
3.5.1 What Should Each State Represent? . . . 12
3.6 FSM State Diagram - Adding Coins . . . 13
3.7 FSM State Diagram - Dispensing Drinks . . . 13
3.8 Step-by-Step Example Scenarios . . . 14
3.8.1 Scenario 1: Buy Rango with Exact Change . . . 14
3.8.2 Scenario 2: Buy Next with Extra Money . . . 14
3.8.3 Scenario 3: Insufficient Funds . . . 15
3.8.4 Scenario 4: Return Money Without Purchase . . . 15
3.9 Complete State Transition Table . . . 15
3.10 Change Calculation Formula . . . 15
3.11 Three-Process FSM Architecture . . . 16
3.12 VHDL Implementation . . . 16
3.13 Testbench . . . 20

4 Why Moore Machine for Vending Machine? **23**
4.1 Key Design Decisions . . . 23

5 Conclusion **24**

1 Introduction to FSM-Based Control

In traditional digital design, Boolean equations directly map inputs to outputs. However, as systems grow in complexity (like a Vending Machine), managing input combinations becomes unmanageable. Instead, we use a **Controller** based on Finite State Machines (FSM).

1.1 Why FSM for Complex Systems?

- **State Memory:** FSM breaks complex problems into "States" (memory) and "Transitions" (logic)
- **Scalability:** Easier to add new features without redesigning entire system
- **Verification:** State diagrams provide visual verification
- **Modularity:** Clear separation between state register, next-state logic, and output logic

1.2 Moore Machine Characteristics

This lab uses **Moore Machines** because:

- Output depends **ONLY** on current state
- Output changes synchronously with state transitions
- More stable outputs (no glitches from input changes)
- Easier to design and debug for complex systems

2 In-Lab Task 1: Multi-Mode Complex Counter (Moore FSM)

2.1 Problem Statement

Design a 3-bit counter that can switch between Binary and Gray Code sequences based on a mode input `m`:

Mode (m)	Sequence Type	Pattern
0	Binary	0, 1, 2, 3, 4, 5, 6, 7, 0, ...
1	Gray Code	0, 1, 3, 2, 6, 7, 5, 4, 0, ...

Table 1: Counter Mode Selection

2.2 Understanding the Problem

Let's break down what we need to build:

- A **3-bit counter** means we have 8 possible values: 0 to 7 (or 000 to 111 in binary)
- The counter has a **mode switch** called `m`
- When `m = 0`: Count in normal binary order (0→1→2→3→4→5→6→7→0...)
- When `m = 1`: Count in Gray code order (0→1→3→2→6→7→5→4→0...)

2.2.1 What is Gray Code?

Gray code is a special counting sequence where **only ONE bit changes** between consecutive numbers. This is useful in hardware to prevent glitches during transitions.

Binary Count	Binary Value	Gray Code Value	Decimal
Step 1	000	000	0
Step 2	001	001	1
Step 3	010	011	3
Step 4	011	010	2
Step 5	100	110	6
Step 6	101	111	7
Step 7	110	101	5
Step 8	111	100	4

Table 2: Binary vs Gray Code Comparison

2.3 Design Approach - Why Moore Machine?

This is a **Moore Machine** because:

- The output (3-bit count value) depends **ONLY** on the current state
- Each state has a **fixed output value** regardless of the mode input

- The mode input `m` only affects **which state comes next**, not what we output

Key Insight: In a Moore machine, we write the output **inside** each state circle. The output never changes while we're in that state.

2.4 FSM State Diagram - Binary Mode (m=0)

First, let's see the **simple binary counting** path:

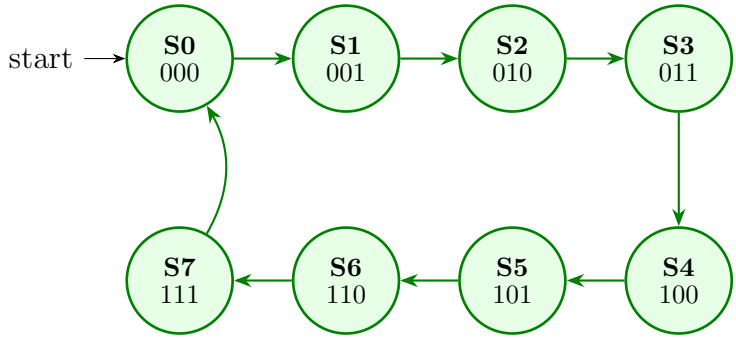

Figure 1: Binary Mode (m=0): Simple Sequential Counting 0→1→2→3→4→5→6→7→0

Explanation: In binary mode, we simply go to the next state in order. S0→S1→S2→S3→S4→S5→ to S0.

2.5 FSM State Diagram - Gray Code Mode (m=1)

Now let's see the **Gray code counting** path (notice the "jumps"):

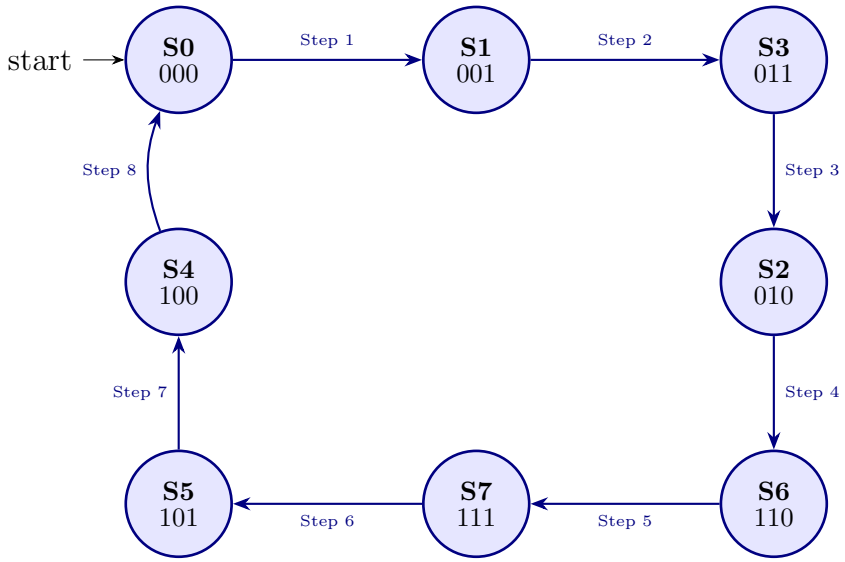

Figure 2: Gray Code Mode (m=1): Non-Sequential Path 0→1→3→2→6→7→5→4→0

Explanation: In Gray code mode, we DON'T go in order! We jump: S0→S1→S3→S2→S6→S7→S5 to S0. This ensures only one bit changes each step.

2.6 Combined FSM State Diagram

Now let's combine both modes into one diagram:

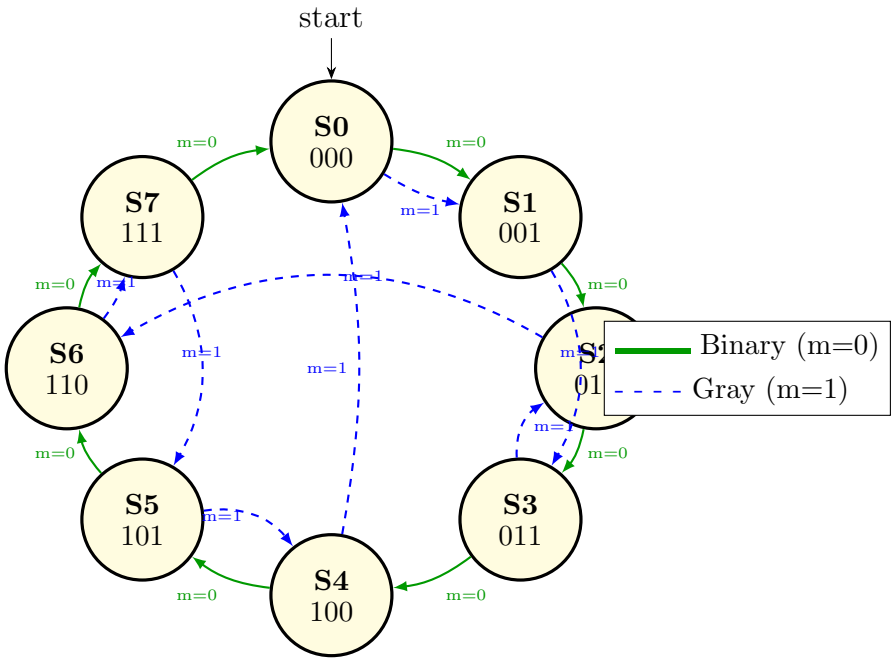

Figure 3: Complete Multi-Mode Counter FSM: Green Solid = Binary, Blue Dashed = Gray Code

2.7 Step-by-Step: How the FSM Works

2.7.1 Example 1: Binary Mode (m=0)

1. Start at **S0** (output = 000)
2. Clock tick → Go to **S1** (output = 001)
3. Clock tick → Go to **S2** (output = 010)
4. Clock tick → Go to **S3** (output = 011)
5. ... and so on in sequence

2.7.2 Example 2: Gray Mode (m=1)

1. Start at **S0** (output = 000)
2. Clock tick → Go to **S1** (output = 001) - bit 0 changed
3. Clock tick → **JUMP** to **S3** (output = 011) - bit 1 changed
4. Clock tick → Go to **S2** (output = 010) - bit 0 changed
5. Clock tick → **JUMP** to **S6** (output = 110) - bit 2 changed
6. ... and so on following Gray code sequence

2.7.3 Example 3: Switching Modes Mid-Count

1. Currently at **S3** in Binary mode (output = 011)
2. User switches mode to Gray (m=1)
3. Clock tick → Go to **S2** (following Gray path from S3)
4. The output is still determined by the state, not the mode!

2.8 State Transition Table

Current State	Output	m=0 (Binary)	m=1 (Gray)
S0	000	S1	S1
S1	001	S2	S3
S2	010	S3	S6
S3	011	S4	S2
S4	100	S5	S0
S5	101	S6	S4
S6	110	S7	S7
S7	111	S0	S5

Table 3: State Transition Table for Multi-Mode Counter

2.9 Gray Code Sequence - Detailed Explanation

The Gray code sequence ensures only one bit changes between consecutive values. Let's trace through each transition:

Step	State	Output	Bit 2	Bit 1	Bit 0
1	S0	000	0	0	0
2	S1	001	0	0	**1** ← changed
3	S3	011	0	**1**	1 ← changed
4	S2	010	0	1	**0** ← changed
5	S6	110	**1**	1	0 ← changed
6	S7	111	1	1	**1** ← changed
7	S5	101	1	**0**	1 ← changed
8	S4	100	1	0	**0** ← changed

Table 4: Gray Code Sequence - Only ONE bit changes per step (shown in red)

Why is this useful?

- In binary counting (0→1→2), when going from 001 to 010, **TWO bits** change simultaneously
- This can cause **glitches** in hardware because bits don't change at exactly the same moment
- Gray code avoids this by ensuring only **ONE bit** ever changes at a time

2.10 VHDL Implementation

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity multi_mode_counter is
    port (
        clk   : in  std_logic;
        rst   : in  std_logic;
        m     : in  std_logic;  -- 0=Binary, 1=Gray
        count : out std_logic_vector(2 downto 0)
    );
end multi_mode_counter;

architecture bhv of multi_mode_counter is
    type state_type is (s0, s1, s2, s3, s4, s5, s6, s7);
    signal current_state, next_state : state_type;
begin

    ------------ Process 1: State Register ------------
    process(clk, rst)
    begin
        if rst = '1' then
            current_state <= s0;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

    ------------ Process 2: Next State Logic ------------
    process(current_state, m)
    begin
        case current_state is
            when s0 =>
                if m = '0' then
                    next_state <= s1;  -- Binary: 0->1
                else
                    next_state <= s1;  -- Gray: 0->1
                end if;
            when s1 =>
                if m = '0' then
                    next_state <= s2;  -- Binary: 1->2
                else
                    next_state <= s3;  -- Gray: 1->3
                end if;
            when s2 =>
                if m = '0' then
                    next_state <= s3;  -- Binary: 2->3
                else
                    next_state <= s6;  -- Gray: 2->6
                end if;
            when s3 =>
                if m = '0' then
                    next_state <= s4;  -- Binary: 3->4
                else
                    next_state <= s2;  -- Gray: 3->2
                end if;
            when s4 =>
```

```
                if m = '0' then
                    next_state <= s5;   -- Binary: 4->5
                else
                    next_state <= s0;   -- Gray: 4->0 (wrap)
                end if;
            when s5 =>
                if m = '0' then
                    next_state <= s6;   -- Binary: 5->6
                else
                    next_state <= s4;   -- Gray: 5->4
                end if;
            when s6 =>
                if m = '0' then
                    next_state <= s7;   -- Binary: 6->7
                else
                    next_state <= s7;   -- Gray: 6->7
                end if;
            when s7 =>
                if m = '0' then
                    next_state <= s0;   -- Binary: 7->0 (wrap)
                else
                    next_state <= s5;   -- Gray: 7->5
                end if;
        end case;
    end process;

    ------------ Process 3: Output Logic (Moore) ------------
    process(current_state)
    begin
        case current_state is
            when s0 => count <= "000";
            when s1 => count <= "001";
            when s2 => count <= "010";
            when s3 => count <= "011";
            when s4 => count <= "100";
            when s5 => count <= "101";
            when s6 => count <= "110";
            when s7 => count <= "111";
        end case;
    end process;

end bhv;
```

Listing 1: multi_mode_counter.vhd

2.11 Testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity multi_mode_counter_tb is
end multi_mode_counter_tb;

architecture sim of multi_mode_counter_tb is
    signal clk   : std_logic := '0';
    signal rst   : std_logic := '0';
    signal m     : std_logic := '0';
```

```
    signal count : std_logic_vector(2 downto 0);

    constant CLK_PERIOD : time := 10 ns;
begin

    -- Instantiate DUT
    DUT: entity work.multi_mode_counter
        port map (
            clk   => clk,
            rst   => rst,
            m     => m,
            count => count
        );

    -- Clock generation
    clk <= not clk after CLK_PERIOD/2;

    -- Stimulus process
    process
    begin
        -- Reset
        rst <= '1';
        wait for 20 ns;
        rst <= '0';

        -- Test Binary mode (m=0)
        m <= '0';
        wait for 100 ns;  -- Count through 0-7

        -- Test Gray mode (m=1)
        m <= '1';
        rst <= '1';
        wait for 10 ns;
        rst <= '0';
        wait for 100 ns;  -- Count through Gray sequence

        -- End simulation
        wait;
    end process;

end sim;
```

Listing 2: multi_mode_counter_tb.vhd

3 In-Lab Task 2: Vending Machine Design (Moore FSM)

3.1 Understanding the Problem

We need to design a **soft-drink dispensing machine** that:

1. Accepts coins and keeps track of total credit
2. Allows user to select a drink
3. Dispenses the drink if enough credit is available
4. Returns the correct change

3.2 Machine Specifications

3.2.1 Accepted Coins

The machine accepts 4 types of coins:

Coin Input	Value	Signal Name
Rs. 1 coin	1	`coin_1`
Rs. 2 coin	2	`coin_2`
Rs. 4 coin	4	`coin_4`
Rs. 8 coin	8	`coin_8`

Table 5: Accepted Coins and Signal Names

3.2.2 Available Drinks

Drink	Price (Rs.)	Selection Signal	Dispense Signal
Rango	3	`sel_rango`	`disp_rango`
Next	4	`sel_next`	`disp_next`
Fizup	5	`sel_fizup`	`disp_fizup`

Table 6: Drink Options with Prices and Signals

3.2.3 Machine Rules

1. **Sufficient credit required:** Customer must deposit enough money before buying
2. **Single coin at a time:** Machine ignores multiple coins inserted simultaneously
3. **Single selection at a time:** Multiple drink selections are ignored
4. **Maximum credit = Rs. 15:** Additional coins beyond this are returned immediately
5. **Coin return:** Pressing ENTER without selection returns all credit as change

3.3 System Block Diagram

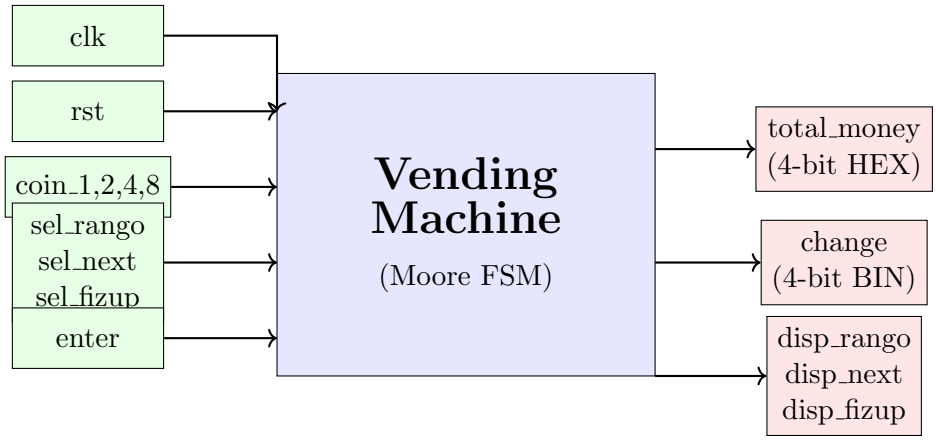

Figure 4: Vending Machine Block Diagram - Inputs and Outputs

3.4 Why Moore Machine?

Feature	**Moore (We Use This)**	**Mealy (Alternative)**
Output depends on	**State ONLY**	State + Input
When output changes	On clock edge only	Immediately with input
Stability	Very stable	Can have glitches
For vending machine	Safe - drink dispenses only after state is confirmed	Risky - noisy input could cause partial dispense

Table 7: Why Moore is Better for Vending Machine

Key Point: In a real vending machine, we don't want the drink dispenser to activate based on a noisy button press. Moore ensures the output is stable and only changes when the state officially changes on a clock edge.

3.5 State Design - The Heart of the FSM

3.5.1 What Should Each State Represent?

Each state represents the **current credit amount** in the machine:

State	Credit	Can Buy	Output (total_money)
S0	Rs. 0	Nothing	0000 (0x0)
S1	Rs. 1	Nothing	0001 (0x1)
S2	Rs. 2	Nothing	0010 (0x2)
S3	Rs. 3	**Rango**	0011 (0x3)
S4	Rs. 4	**Rango, Next**	0100 (0x4)
S5	Rs. 5	**All drinks**	0101 (0x5)
S6	Rs. 6	**All drinks**	0110 (0x6)
S7	Rs. 7	**All drinks**	0111 (0x7)
S8	Rs. 8	**All drinks**	1000 (0x8)
...	...	...	...
S15	Rs. 15	**All drinks (MAX)**	1111 (0xF)

Table 8: All 16 States and What Can Be Purchased (Green = Can Buy Something)

3.6 FSM State Diagram - Adding Coins

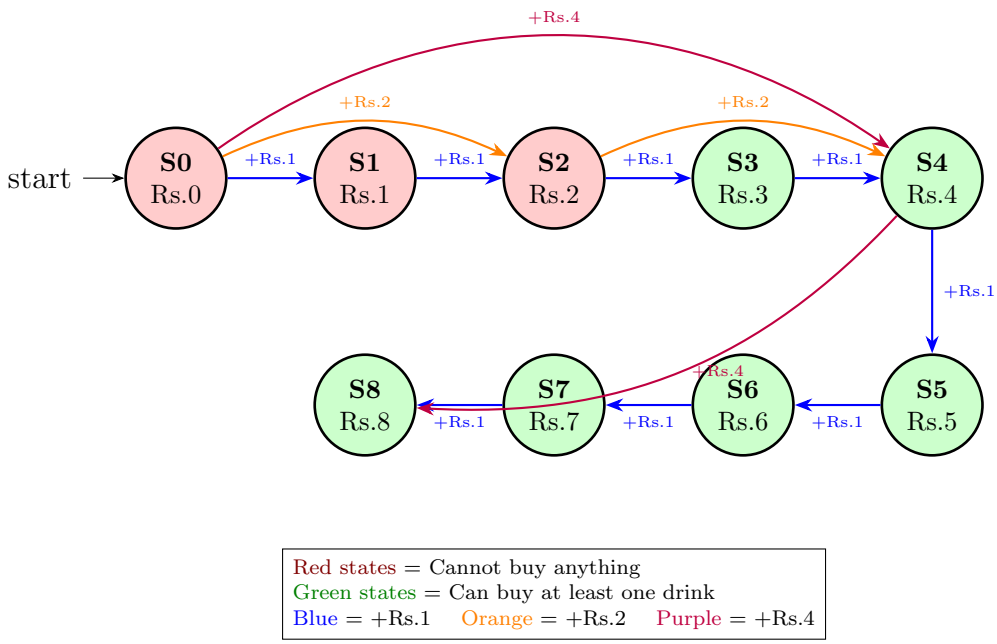

Figure 5: FSM Coin Addition - How Credit Increases (Partial View)

3.7 FSM State Diagram - Dispensing Drinks

When ENTER is pressed with a drink selected, the machine dispenses and returns to a lower state:

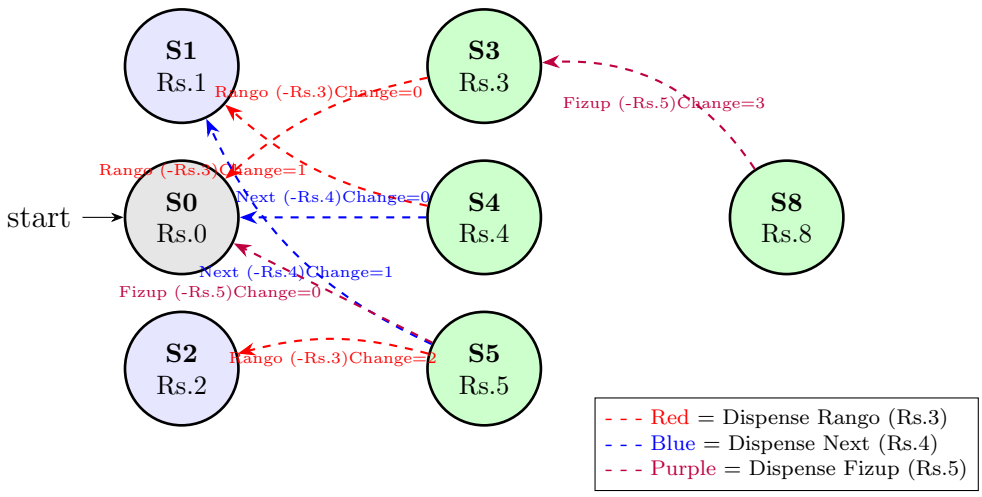

Figure 6: FSM Drink Dispensing - State Transitions with Change Calculation

3.8 Step-by-Step Example Scenarios

3.8.1 Scenario 1: Buy Rango with Exact Change

1. **Start:** Machine in S0 (Rs. 0 credit)
2. **Insert Rs. 2 coin:** Machine transitions to S2 (Rs. 2 credit)
3. **Insert Rs. 1 coin:** Machine transitions to S3 (Rs. 3 credit)
4. **Select Rango + Press ENTER:**
   - Credit (3) $\geq$ Rango price (3)
   - Dispense Rango: `disp_rango = 1`
   - Change = 3 - 3 = 0: `change = 0000`
   - Transition to S0

3.8.2 Scenario 2: Buy Next with Extra Money

1. **Start:** Machine in S0 (Rs. 0 credit)
2. **Insert Rs. 8 coin:** Machine transitions to S8 (Rs. 8 credit)
3. **Select Next + Press ENTER:**
   - Credit (8) $\geq$ Next price (4)
   - Dispense Next: `disp_next = 1`
   - Change = 8 - 4 = 4: `change = 0100`
   - Transition to S4 (remaining credit)
4. **Note:** User still has Rs. 4 credit! Can buy another drink or press ENTER to get change.

3.8.3 Scenario 3: Insufficient Funds

1. **Start:** Machine in S0
2. **Insert Rs. 2 coin:** Machine transitions to S2 (Rs. 2 credit)
3. **Select Fizup (Rs. 5) + Press ENTER:**
   - Credit (2) < Fizup price (5) ×
   - **Nothing dispensed!**
   - Machine stays in S2, waiting for more coins

3.8.4 Scenario 4: Return Money Without Purchase

1. **Start:** Machine in S0
2. **Insert Rs. 4 coin:** Machine transitions to S4
3. **Insert Rs. 2 coin:** Machine transitions to S6 (Rs. 6 credit)
4. **Press ENTER without selecting drink:**
   - No drink selected
   - Return all credit as change: `change = 0110` (6 in binary)
   - Transition to S0

3.9 Complete State Transition Table

Current State	**+Rs.1**	**+Rs.2**	**+Rs.4**	**+Rs.8**	**Rango (-Rs.3)**	**Next (-Rs.4)**	**Fizup (-Rs.5)**	**Output**
S0 (Rs.0)	S1	S2	S4	S8	–	–	–	0000
S1 (Rs.1)	S2	S3	S5	S9	–	–	–	0001
S2 (Rs.2)	S3	S4	S6	S10	–	–	–	0010
S3 (Rs.3)	S4	S5	S7	S11	S0	–	–	0011
S4 (Rs.4)	S5	S6	S8	S12	S1	S0	–	0100
S5 (Rs.5)	S6	S7	S9	S13	S2	S1	S0	0101
S6 (Rs.6)	S7	S8	S10	S14	S3	S2	S1	0110
S7 (Rs.7)	S8	S9	S11	S15	S4	S3	S2	0111
S8 (Rs.8)	S9	S10	S12	S15*	S5	S4	S3	1000
...	...	...	...	...	...	...	...	...
S15 (Rs.15)	S15*	S15*	S15*	S15*	S12	S11	S10	1111

Table 9: State Transition Table (* = excess coins returned, – = insufficient funds)

3.10 Change Calculation Formula

$$\text{Change} = \text{Current Credit} - \text{Drink Price} \tag{1}$$

If Credit =	Buy Rango (Rs.3)	Buy Next (Rs.4)	Buy Fizup (Rs.5)
Rs. 3	Change = 0	Cannot buy	Cannot buy
Rs. 4	Change = 1	Change = 0	Cannot buy
Rs. 5	Change = 2	Change = 1	Change = 0
Rs. 6	Change = 3	Change = 2	Change = 1
Rs. 7	Change = 4	Change = 3	Change = 2
Rs. 8	Change = 5	Change = 4	Change = 3

Table 10: Change Calculation Examples

3.11 Three-Process FSM Architecture

The VHDL code follows the standard **three-process** Moore FSM pattern:

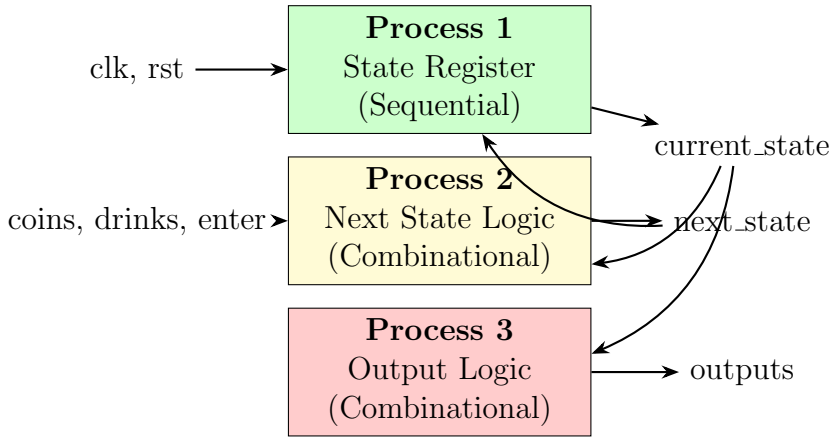

Figure 7: Three-Process FSM Architecture

3.12 VHDL Implementation

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity vending_machine is
    port (
        clk          : in  std_logic;
        rst          : in  std_logic;
        -- Coin inputs (active high, only one at a time)
        coin_1       : in  std_logic;  -- Rs. 1
        coin_2       : in  std_logic;  -- Rs. 2
        coin_4       : in  std_logic;  -- Rs. 4
        coin_8       : in  std_logic;  -- Rs. 8
        -- Drink selection (active high, only one at a time)
        sel_rango    : in  std_logic;  -- Rs. 3
        sel_next     : in  std_logic;  -- Rs. 4
        sel_fizup    : in  std_logic;  -- Rs. 5
        -- Control
        enter        : in  std_logic;
        -- Outputs
        total_money  : out std_logic_vector(3 downto 0);  -- HEX
        change       : out std_logic_vector(3 downto 0);  -- Binary
```

```
        disp_rango  : out std_logic;
        disp_next   : out std_logic;
        disp_fizup  : out std_logic
    );
end vending_machine;

architecture bhv of vending_machine is
    -- States represent credit amount (0 to 15)
    type state_type is (s0, s1, s2, s3, s4, s5, s6, s7,
                        s8, s9, s10, s11, s12, s13, s14, s15);
    signal current_state, next_state : state_type;

    -- Internal signals
    signal credit : integer range 0 to 15;
    signal coin_value : integer range 0 to 8;
    signal drink_price : integer range 0 to 5;
    signal change_amount : integer range 0 to 15;

begin

    ------------ Coin Value Decoder ------------
    coin_value <= 1 when coin_1 = '1' else
                  2 when coin_2 = '1' else
                  4 when coin_4 = '1' else
                  8 when coin_8 = '1' else
                  0;

    ------------ Drink Price Decoder ------------
    drink_price <= 3 when sel_rango = '1' else
                   4 when sel_next = '1' else
                   5 when sel_fizup = '1' else
                   0;

    ------------ Process 1: State Register ------------
    process(clk, rst)
    begin
        if rst = '1' then
            current_state <= s0;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

    ------------ Process 2: Next State Logic ------------
    process(current_state, coin_value, drink_price, enter)
        variable new_credit : integer range 0 to 23;
    begin
        next_state <= current_state;  -- Default: stay

        -- State to credit mapping
        case current_state is
            when s0  => credit <= 0;
            when s1  => credit <= 1;
            when s2  => credit <= 2;
            when s3  => credit <= 3;
            when s4  => credit <= 4;
            when s5  => credit <= 5;
            when s6  => credit <= 6;
```

```
            when s7  => credit <= 7;
            when s8  => credit <= 8;
            when s9  => credit <= 9;
            when s10 => credit <= 10;
            when s11 => credit <= 11;
            when s12 => credit <= 12;
            when s13 => credit <= 13;
            when s14 => credit <= 14;
            when s15 => credit <= 15;
        end case;

        -- Add coin (if not at max)
        if coin_value > 0 and enter = '0' then
            new_credit := credit + coin_value;
            if new_credit > 15 then
                new_credit := 15;  -- Cap at max, return excess
            end if;
            -- Transition to new state
            case new_credit is
                when 0  => next_state <= s0;
                when 1  => next_state <= s1;
                when 2  => next_state <= s2;
                when 3  => next_state <= s3;
                when 4  => next_state <= s4;
                when 5  => next_state <= s5;
                when 6  => next_state <= s6;
                when 7  => next_state <= s7;
                when 8  => next_state <= s8;
                when 9  => next_state <= s9;
                when 10 => next_state <= s10;
                when 11 => next_state <= s11;
                when 12 => next_state <= s12;
                when 13 => next_state <= s13;
                when 14 => next_state <= s14;
                when 15 => next_state <= s15;
                when others => next_state <= s15;
            end case;
        end if;

        -- Dispense drink (when ENTER pressed)
        if enter = '1' then
            if drink_price > 0 and credit >= drink_price then
                -- Calculate remaining credit
                new_credit := credit - drink_price;
                case new_credit is
                    when 0  => next_state <= s0;
                    when 1  => next_state <= s1;
                    when 2  => next_state <= s2;
                    when 3  => next_state <= s3;
                    when 4  => next_state <= s4;
                    when 5  => next_state <= s5;
                    when 6  => next_state <= s6;
                    when 7  => next_state <= s7;
                    when 8  => next_state <= s8;
                    when 9  => next_state <= s9;
                    when 10 => next_state <= s10;
                    when others => next_state <= s0;
                end case;
```

```
            elsif drink_price = 0 then
                -- No drink selected, return all money
                next_state <= s0;
            end if;
        end if;
    end process;

    ------------ Process 3: Output Logic (Moore) ------------
    process(current_state, enter, drink_price, credit)
    begin
        -- Default outputs
        disp_rango <= '0';
        disp_next  <= '0';
        disp_fizup <= '0';
        change     <= "0000";

        -- Total money display (Moore output - depends on state)
        case current_state is
            when s0  => total_money <= "0000";
            when s1  => total_money <= "0001";
            when s2  => total_money <= "0010";
            when s3  => total_money <= "0011";
            when s4  => total_money <= "0100";
            when s5  => total_money <= "0101";
            when s6  => total_money <= "0110";
            when s7  => total_money <= "0111";
            when s8  => total_money <= "1000";
            when s9  => total_money <= "1001";
            when s10 => total_money <= "1010";
            when s11 => total_money <= "1011";
            when s12 => total_money <= "1100";
            when s13 => total_money <= "1101";
            when s14 => total_money <= "1110";
            when s15 => total_money <= "1111";
        end case;

        -- Dispense and change (when ENTER is pressed)
        if enter = '1' then
            if credit >= 3 and sel_rango = '1' then
                disp_rango <= '1';
                change <= std_logic_vector(to_unsigned(credit - 3, 4));
            elsif credit >= 4 and sel_next = '1' then
                disp_next <= '1';
                change <= std_logic_vector(to_unsigned(credit - 4, 4));
            elsif credit >= 5 and sel_fizup = '1' then
                disp_fizup <= '1';
                change <= std_logic_vector(to_unsigned(credit - 5, 4));
            else
                -- Return all money as change
                change <= std_logic_vector(to_unsigned(credit, 4));
            end if;
        end if;
    end process;

end bhv;
```

Listing 3: vending_machine.vhd

3.13 Testbench

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity vending_machine_tb is
end vending_machine_tb;

architecture sim of vending_machine_tb is
    signal clk          : std_logic := '0';
    signal rst          : std_logic := '0';
    signal coin_1       : std_logic := '0';
    signal coin_2       : std_logic := '0';
    signal coin_4       : std_logic := '0';
    signal coin_8       : std_logic := '0';
    signal sel_rango    : std_logic := '0';
    signal sel_next     : std_logic := '0';
    signal sel_fizup    : std_logic := '0';
    signal enter        : std_logic := '0';
    signal total_money  : std_logic_vector(3 downto 0);
    signal change       : std_logic_vector(3 downto 0);
    signal disp_rango   : std_logic;
    signal disp_next    : std_logic;
    signal disp_fizup   : std_logic;

    constant CLK_PERIOD : time := 10 ns;
begin

    -- Instantiate DUT
    DUT: entity work.vending_machine
        port map (
            clk          => clk,
            rst          => rst,
            coin_1       => coin_1,
            coin_2       => coin_2,
            coin_4       => coin_4,
            coin_8       => coin_8,
            sel_rango    => sel_rango,
            sel_next     => sel_next,
            sel_fizup    => sel_fizup,
            enter        => enter,
            total_money  => total_money,
            change       => change,
            disp_rango   => disp_rango,
            disp_next    => disp_next,
            disp_fizup   => disp_fizup
        );

    -- Clock generation
    clk <= not clk after CLK_PERIOD/2;

    -- Stimulus process
    process
    begin
        -- Reset
        rst <= '1';
        wait for 20 ns;
```

```
        rst <= '0';
        wait for 10 ns;

        -- Test Case 1: Buy Rango (Rs.3) with Rs.4
        -- Insert Rs.4 coin
        coin_4 <= '1';
        wait for CLK_PERIOD;
        coin_4 <= '0';
        wait for CLK_PERIOD;

        -- Select Rango and press Enter
        sel_rango <= '1';
        enter <= '1';
        wait for CLK_PERIOD;
        sel_rango <= '0';
        enter <= '0';
        wait for 20 ns;

        -- Test Case 2: Buy Fizup (Rs.5) with Rs.8
        rst <= '1';
        wait for 10 ns;
        rst <= '0';
        wait for 10 ns;

        -- Insert Rs.8 coin
        coin_8 <= '1';
        wait for CLK_PERIOD;
        coin_8 <= '0';
        wait for CLK_PERIOD;

        -- Select Fizup and press Enter
        sel_fizup <= '1';
        enter <= '1';
        wait for CLK_PERIOD;
        sel_fizup <= '0';
        enter <= '0';
        wait for 20 ns;

        -- Test Case 3: Return money without selection
        rst <= '1';
        wait for 10 ns;
        rst <= '0';
        wait for 10 ns;

        -- Insert Rs.2 coins twice (total Rs.4)
        coin_2 <= '1';
        wait for CLK_PERIOD;
        coin_2 <= '0';
        wait for CLK_PERIOD;
        coin_2 <= '1';
        wait for CLK_PERIOD;
        coin_2 <= '0';
        wait for CLK_PERIOD;

        -- Press Enter without selection
        enter <= '1';
        wait for CLK_PERIOD;
        enter <= '0';
```

```
        wait for 20 ns;

        -- End simulation
        wait;
    end process;

end sim;
```

Listing 4: vending_machine_tb.vhd

4 Why Moore Machine for Vending Machine?

Aspect	Moore (Used)	Mealy (Alternative)
Output depends on	Current state only	State + Input
Output timing	Changes on clock edge	Changes immediately with input
For vending machine	More stable - prevents glitches in mechanical dispensing	Could cause partial dispensing on noisy inputs
Number of states	16 states (Rs. 0-15)	Could be fewer but more complex
Debugging	Easier - output predictable from state	Harder - output depends on input timing

Table 11: Moore vs Mealy for Vending Machine

4.1 Key Design Decisions

1. **State per Credit Value:** Each state represents a specific credit amount, making the design intuitive and easy to verify
2. **Single Coin Detection:** Multiple coin inputs are ignored (only first detected coin counts)
3. **Max Credit Cap:** Credit caps at Rs. 15; additional coins are returned
4. **Three-Process Architecture:**
   - Process 1: State register (sequential)
   - Process 2: Next state logic (combinational)
   - Process 3: Output logic (combinational, Moore-style)

5 Conclusion

This lab demonstrated:

1. **Multi-Mode Counter:** Using FSM to implement a counter that switches between Binary and Gray code sequences based on mode input
2. **Vending Machine Controller:** Using Moore FSM to manage:
   - Credit accumulation (coin insertion)
   - Drink selection and dispensing
   - Change calculation and return
3. **Moore Machine Benefits:**
   - Output stability (changes only on clock edges)
   - Easier debugging (output determined solely by state)
   - Suitable for physical systems requiring stable control signals
4. **FSM Design Pattern:** Three-process architecture provides clean separation between:
   - State memory (register)
   - State transition logic
   - Output generation